

Relatório do Projeto de Programação e Análise de Algoritmos-2026/1

Luciana Alves Pereira, 23100348

Anna Luiza Novaes Jansen Silva, 232024572

June 24, 2026

1 Introdução

Este projeto teve como principal objetivo aplicar os conceitos de lógica que estudamos em aula para formalizar e provar, no assistente de provas Coq, a equivalência entre duas definições distintas de permutação de listas de números naturais.

No início, escolhemos a Proposta 4, sobre a correção do algoritmo de ordenação por inserção binária, pois achamos o tema interessante e encontramos alguns materiais que pareciam ajudar. Porém, ao começar a desenvolver as provas, percebemos que a complexidade estava além do que conseguíamos avançar no tempo disponível. A função que realiza a busca binária para encontrar a posição de inserção exigiu uma quantidade grande de lemas auxiliares só para provar propriedades básicas, e tivemos dificuldade em fechar até mesmo os casos mais simples. Diante disso, decidimos migrar para a Proposta 7, sobre equivalência entre noções de permutação, que achamos que seria mais fácil.

O trabalho está dividido em duas partes, mostrar que a definição indutiva de permutação implica a equivalência por contagem de ocorrências, e a direção contrária. Ao longo do relatório, explicamos todas as etapas do processo, desde as dificuldades iniciais com a ferramenta até a finalização das provas.

2 Prova do Algoritmo

2.1 Primeira Etapa - Permutation para equiv

A primeira etapa consistiu em provar que toda permutação no sentido indutivo implica que as duas listas possuem o mesmo número de ocorrências de qualquer elemento. Essa direção foi mais tranquila de desenvolver, pois a definição de permutação usada é indutiva e tem quatro casos bem definidos, sendo

eles: o caso da lista vazia, o caso em que se adiciona o mesmo elemento no início de duas listas já permutadas, o caso em que dois elementos adjacentes são trocados, e o caso de transitividade.

Como a prova segue por indução na derivação da permutação, cada caso foi tratado separadamente. Em todos eles, bastou mostrar que a contagem de ocorrências de qualquer elemento se mantinha igual após a operação. Os casos de trocar elementos adjacentes e de adicionar o mesmo elemento no início foram os mais diretos, pois bastou analisar os casos booleanos da comparação entre naturais. O caso transitivo foi resolvido encadeando os resultados das duas hipóteses de indução. Essa etapa ajudou bastante a entender como a definição indutiva funciona na prática e deu mais confiança para enfrentar a etapa seguinte.

2.2 Segunda Etapa de Correção do Algoritmo

A segunda etapa foi bem mais difícil e foi onde concentramos a maior parte do esforço. O objetivo era provar a direção contrária, que se duas listas possuem o mesmo número de ocorrências de qualquer elemento, então elas são permutações uma da outra no sentido indutivo. O desafio central é que a hipótese é uma afirmação sobre contagens, e transformar isso em uma permutação concreta não foi nada fácil.

A estratégia que encontramos foi fazer indução na estrutura da primeira lista. O caso base foi simples, se uma lista vazia tem as mesmas contagens que outra lista, essa outra lista também precisa ser vazia, o que foi provado com um lema auxiliar específico para esse caso. Para o caso indutivo, separamos dois subcasos dependendo se os primeiros elementos das duas listas são iguais ou diferentes. Quando os primeiros elementos são iguais, criamos um lema auxiliar que garante que, se duas listas com o mesmo primeiro el-

emento têm as mesmas contagens, então suas caudas também têm as mesmas contagens. Com isso, foi possível aplicar a hipótese de indução diretamente para o restante das listas.

Quando os primeiros elementos são diferentes, o trabalho foi maior. Precisamos primeiro localizar onde o primeiro elemento da lista original aparece na segunda lista, para então reorganizá-la. Para isso, criamos um lema que mostra que, se uma lista não vazia tem as mesmas contagens que outra lista, então o elemento que está na cabeça da primeira aparece em alguma posição da segunda, o que nos permite dividi-la em duas partes em torno desse elemento. Com isso, usamos um resultado da biblioteca padrão do Coq que permite mover um elemento do meio de uma lista para o início, transformando o problema em algo que a hipótese de indução consegue resolver.

Além disso, foi necessário criar dois lemas adicionais, um que mostra como a contagem de ocorrências se distribui sobre a concatenação de duas listas, e outro que usa a comutatividade da adição para trocar a ordem de duas sublistas sem alterar as contagens.

2.3 Aprendizado Prático

Apesar das dificuldades, o projeto trouxe bastante aprendizado. O principal foi entender na prática como a lógica formal funciona aplicada a propriedades reais, e perceber que provar a equivalência entre duas definições exige muito trabalho, já que cada caso precisa ser tratado separadamente.

Aprendemos também que criar lemas auxiliares é essencial para conseguir avançar em provas mais complexas. No início tentamos resolver tudo de uma vez e não conseguimos, e foi só quando começamos a quebrar o problema em partes menores que as coisas foram se encaixando. Isso foi um aprendizado que vai além do Coq em si.

O contato com a ferramenta também mudou um pouco nossa visão sobre o que significa provar que algo está correto. No dia a dia da programação, a gente testa e assume que funciona, mas no Coq cada detalhe precisa ser formalmente justificado, e isso faz pensar de um jeito diferente sobre o que é uma prova de verdade.

2.4 Experiência no Desenvolvimento

O desenvolvimento do projeto apresentou vários desafios, principalmente para adaptar o raciocínio matemático às exigências específicas do Coq. Muitas

propriedades exigiram provas mais detalhadas o que deixou mais complexo.

Um dos primeiros obstáculos foi a própria configuração do ambiente. Instalar e configurar o Coq para funcionar corretamente no VS Code demorou bastante, e também tivemos alguns problemas de compatibilidade. Depois de resolver a parte de configuração, ainda foi preciso aprender a lidar com a ferramenta em si, já que o Coq tem um jeito muito específico de funcionar e nem sempre as mensagens de erro ajudam a entender o que está errado.

Lidar com igualdades envolvendo comparações booleanas entre naturais foi trabalhoso, pois o Coq exige que esses casos sejam tratados de forma explícita e separada. Algumas táticas que pareciam fazer sentido simplesmente não funcionavam do jeito esperado, e precisamos experimentar bastante até encontrar a combinação certa. Buscamos ajuda em projetos no GitHub para entender melhor as estratégias de prova, o que foi importante para avançar em momentos em que ficávamos travados.

No geral, o projeto foi útil para exercitar o pensamento lógico e a construção de provas formais.

3 Conclusão

Um dos principais aprendizados deste projeto foi entender a importância de dividir um problema complexo em partes menores. Ao lidar com a prova da equivalência entre as duas definições de permutação, ficou claro que tentar resolver tudo de uma vez tornaria o processo inviável. A criação dos lemmas auxiliares foi muito importante, pois a partir disso conseguimos tratar as situações.

Com isso, conseguimos finalizar a prova do algoritmo. A experiência mostrou, na prática, como a lógica formal e a decomposição de problemas são ferramentas importantes para garantir a correção de propriedades sobre estruturas de dados.